Министерство образования Республики Беларусь

Учреждение образования

«Брестский Государственный технический университет»

Кафедра ИИТ

Лабораторная работа №1

По дисциплине «КиСБ»

Тема: «Исследование алгоритмов генерации псевдослучайных последовательностей и методов тестирования чисел на простоту»

Выполнил:
Студент 4 курса
Группы АС-66
Лысюк Р. А.
Проверила:
Хацкевич А. С.

Брест 2026

Цель работы: освоить программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.

Ход работы

Общая схема выполнения (Индивидуальный вариант): задание выполняется строго в соответствии со своим вариантом (номер варианта выбирается по списку подгруппы от 1 до 15). Все задачи объединены в единый программный комплекс (консольное приложение).

1. Генерация и тестирование псевдослучайных последовательностей (файл "Лабораторная работа №1 (ч.1)")

В рамках данного этапа необходимо разработать программный модуль генерации последовательностей и провести их первичный анализ.

Требования:

- реализовать алгоритм генерации по варианту в виде программной функции/класса, способной выдавать последовательность произвольной длины;
- подобрать параметры генератора (a, c, N, x_0 и др.) так, чтобы добиться максимального периода последовательности T_{max}(согласно теоретическим условиям из методички);
- провести базовое статистическое тестирование полученной последовательности (например, оценить частотность появления элементов, равномерность распределения).

2. Тестирование на простоту и генерация больших простых чисел (файл "Лабораторная работа №1 (ч.2)")

На втором этапе сгенерированные на первом этапе псевдослучайные числа (или базовые механизмы генерации) интегрируются в модуль поиска и проверки чисел на простоту.

Требования:

- реализовать стандартную схему генерации кандидата в простые числа:
    - сгенерировать случайное p-битовое число n(используя генератор из Этапа 1);
    - установить старший и младший биты в 1 (для гарантии разрядности и нечетности);
    - выполнить предварительную фильтрацию (просеивание) делением на малые простые числа (2, 3, 5, 7, ..., вплоть до 256 или 2000), что отсекает до 80% составных чисел без тяжелых тестов;

- интегрировать целевой тест из таблицы вариантов (вместо стандартного Рабина-Миллера или в дополнение к нему) для финальной проверки кандидата. (Примечание: для метода Михалеску используется собственная специфическая схема генерации доказуемо простых чисел).

Отчет должен содержать:

1. Постановочную часть по обоим этапам для выбранного варианта.
2. Листинги ключевых фрагментов разработанного программного обеспечения.
3. Графики и таблицы с результатами тестирования: демонстрацию изменения свойств сгенерированных последовательностей при изменении параметров генератора (Этап 1), а также лог работы генератора простых чисел с фиксацией времени и количества отсеянных кандидатов (Этап 2).

Вариант 4:

4	LFSR		*Рассмотреть следующие случаи генераторов, основанных на примитивных многочленах:* 1. $f(x) = x^7 + x + 1$ 2. $f(x) = x^7 + x^5 + x^3 + 1$ 3. $f(x) = x^7 + x^6 + x^5 + x^2 + 1$ *Какие из этих многочленов примитивны по модулю 2?*

4	Тест Рабина – Миллера

Текст программы:

```
import sys

import random

import time


if hasattr(sys.stdout, "reconfigure"):

    sys.stdout.reconfigure(encoding="utf-8")


def deg(p):

    return p.bit_length() - 1 if p else -1


def modp(a, m):

    dm = deg(m)

    while a and deg(a) >= dm:

        a ^= m << (deg(a) - dm)

    return a
```

```
def mulp(a, b, m):
    r = 0
    while b:
        if b & 1:
            r ^= a
        b >>= 1
        a <<= 1
    return modp(r, m)

def powp(b, e, m):
    r = 1
    b = modp(b, m)
    while e:
        if e & 1:
            r = mulp(r, b, m)
        b = mulp(b, b, m)
        e >>= 1
    return r

def gcdp(a, b):
    while b:
        a, b = b, modp(a, b)
    return a

def divisors(n):
    s = set()
    d = 2
    while d * d <= n:
        while n % d == 0:
            s.add(d)
            n //= d
        d += 1
    if n > 1:
        s.add(n)
    return s

def irreducible(f):
    n = deg(f)
    if n <= 0 or (f & 1) == 0:
        return False
```

```
    x = 0b10
    for d in divisors(n):
        if d == n:
            continue
        if gcdp(powp(x, 1 << d, f) ^ x, f) != 1:
            return False
    return powp(x, 1 << n, f) == x


def primitive(f):
    if not irreducible(f):
        return False
    n = deg(f)
    order = (1 << n) - 1
    x = 0b10
    if powp(x, order, f) != 1:
        return False
    t = order
    d = 2
    while d * d <= t:
        if t % d == 0:
            if powp(x, order // d, f) == 1:
                return False
            while t % d == 0:
                t //= d
        d += 1
    if t > 1 and powp(x, order // t, f) == 1:
        return False
    return True


class Shift:
    def __init__(self, taps, n, seed):
        self.taps = taps
        self.n = n
        self.seed = seed & ((1 << n) - 1)
        if self.seed == 0:
            self.seed = 1
        self.state = self.seed

    def bit(self):
        fb = 0
```

```
        for i in range(self.n):
            if (self.taps >> i) & 1:
                fb ^= (self.state >> i) & 1
        out = self.state & 1
        self.state = (self.state >> 1) | (fb << (self.n - 1))
        return out

    def word(self):
        w = 0
        for _ in range(32):
            w = (w << 1) | self.bit()
        return w

    def bits(self, count):
        return [self.bit() for _ in range(count)]

    def period(self):
        init = self.state
        limit = (1 << self.n) - 1
        t = 0
        while True:
            self.bit()
            t += 1
            if self.state == init or t > limit:
                break
        return t

def freq(bits):
    n = len(bits)
    o = sum(bits)
    z = n - o
    e = n / 2
    c = (o - e) ** 2 / e + (z - e) ** 2 / e
    return o, z, c, 3.841, c < 3.841

def stage1():
    cands = [
        ("f1(x) = x^7 + x + 1",
         (1 << 7) | (1 << 1) | 1,
         0b0000011),
```

```
        ("f2(x) = x^7 + x^5 + x^3 + 1",
         (1 << 7) | (1 << 5) | (1 << 3) | 1,
         0b0101001),
        ("f3(x) = x^7 + x^6 + x^5 + x^2 + 1",
         (1 << 7) | (1 << 6) | (1 << 5) | (1 << 2) | 1,
         0b01100101),
    ]
    print("Primitivity of degree-7 polynomials over GF(2):")
    print(f"{'polynomial':42s} {'irreducible':12s} {'primitive':12s} {'period':>8s}")
    good = []
    for name, poly, taps in cands:
        irr = irreducible(poly)
        prm = primitive(poly) if irr else False
        g = Shift(taps, 7, seed=0b1010101)
        per = g.period()
        if prm:
            good.append(name)
        print(f"{name:42s} {str(irr):12s} {str(prm):12s} {per:>8d}")
    print("\nAnswer: primitive polynomials modulo 2 - " + (", ".join(good) if good else "none"))
    print("\nFrequency test on bits (20000 bits):")
    print(f"{'polynomial':42s} {'ones':>6s} {'zeros':>6s} {'chi2':>8s} {'crit':>6s} {'uniform':>8s}")
    for name, poly, taps in cands:
        if not primitive(poly):
            continue
        g = Shift(taps, 7, seed=0b1010101)
        b = g.bits(20000)
        o, z, c, cc, u = freq(b)
        print(f"{name:42s} {o:6d} {z:6d} {c:8.3f} {cc:6.3f} {str(u):>8s}")


def sieve(limit):
    bs = bytearray([1]) * (limit + 1)
    bs[0] = bs[1] = 0
    for i in range(2, int(limit ** 0.5) + 1):
        if bs[i]:
            bs[i * i: limit + 1: i] = bytearray(len(range(i * i, limit + 1, i)))
    return [i for i, v in enumerate(bs) if v]


SMALL = sieve(2000)


def rabin(n, rounds, rng):
```

```
    if n < 2:
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False
    for p in SMALL:
        if p * p > n:
            break
        if n == p:
            return True
        if n % p == 0:
            return False
    d = n - 1
    s = 0
    while d % 2 == 0:
        d //= 2
        s += 1
    for _ in range(rounds):
        a = rng.randrange(2, n - 1)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(s - 1):
            x = (x * x) % n
            if x == n - 1:
                break
        else:
            return False
    return True


def draw(p, g):
    n = 0
    got = 0
    while got < p:
        w = g.word()
        take = min(32, p - got)
        n = (n << take) | (w & ((1 << take) - 1))
        got += take
    n |= (1 << (p - 1))
```

```
    n |= 1
    return n

def only_small(n):
    for p in SMALL:
        if p * p > n:
            break
        if n == p:
            return True
        if n % p == 0:
            return False
    return True

def make_prime(pbits, rounds=5):
    g = Shift(0b0000011, 7, seed=random.getrandbits(7) | 1)
    rng = random.Random()
    cands = 0
    out = 0
    tested = 0
    t0 = time.perf_counter()
    while True:
        n = draw(pbits, g)
        cands += 1
        if not only_small(n):
            out += 1
            continue
        tested += 1
        if rabin(n, rounds, rng):
            ms = (time.perf_counter() - t0) * 1000
            return n, cands, out, tested, ms

if __name__ == "__main__":
    random.seed(20241018)
    stage1()
    print("\nPrime generation (Miller-Rabin, 5 rounds):")
    print(f"{'bits':>4s} {'candidates':>11s} {'sieved':>8s} {'tested':>8s} {'ms':>8s}  prime")
    for pbits in (16, 32, 48, 64):
        n, c, o, t, ms = make_prime(pbits)
        print(f"{pbits:>4d} {c:>11d} {o:>8d} {t:>8d} {ms:>8.2f}  {n}")
```

Результат работы программы:

```
[Running] python -u "c:\Users\lysyu\OneDrive\Desktop\КиСБ\KiSB-2026-autumn\lab1\Лысюк\lab1.py"
Primitivity of degree-7 polynomials over GF(2):
polynomial                              irreducible  primitive      period
f1(x) = x^7 + x + 1                     True         True             127
f2(x) = x^7 + x^5 + x^3 + 1             False        False             63
f3(x) = x^7 + x^6 + x^5 + x^2 + 1       True         True             127

Answer: primitive polynomials modulo 2 - f1(x) = x^7 + x + 1, f3(x) = x^7 + x^6 + x^5 + x^2 + 1

Frequency test on bits (20000 bits):
polynomial                               ones  zeros     chi2   crit  uniform
f1(x) = x^7 + x + 1                     10076   9924    1.155  3.841     True
f3(x) = x^7 + x^6 + x^5 + x^2 + 1       10079   9921    1.248  3.841     True

Prime generation (Miller-Rabin, 5 rounds):
bits  candidates   sieved   tested       ms  prime
  16          3        2        1     0.07  47251
  32          4        2        2     0.17  3881035889
  48         14       12        2     0.56  227220159392873
  64         85       78        7     3.10  12173842128367920347

[Done] exited with code=0 in 0.083 seconds
```

Вывод: В ходе работы реализованы LFSR на примитивных многочленах степени 7 над GF(2) и генератор простых чисел с тестом Рабина–Миллера. Установлено, что примитивны $x^7 + x + 1$ и $x^7 + x^6 + x^5 + x^2 + 1$ (период 127), а $x^7 + x^5 + x^3 + 1$ — нет; частотный тест подтвердил равномерность. Просеивание до 2000 отсекает около 80% составных кандидатов до теста Рабина–Миллера, что ускоряет генерацию простых чисел.